

AGENDEAQUI

Scheduling-as-a-Service API

Planejamento de Arquitetura Técnica

Estrutura de Camadas, Conexões Externas e Decisões de Design

Time de Agentes Especialistas

Arquitetura • .NET • QA • PostgreSQL • Mensageria • Disponibilidade

Março 2026 | Documento Técnico | Confidencial

1. Visão Geral da Arquitetura

Este documento consolida as recomendações de seis agentes especialistas para definir a arquitetura técnica da API AgendaAqui. O objetivo é planejar uma API de Scheduling-as-a-Service que seja independente, multi-tenant, e conectável por qualquer sistema externo.

1.1. Princípios Arquiteturais

- Clean Architecture: camadas internas nunca dependem de camadas externas; dependência sempre via interfaces
- DDD (Domain-Driven Design): modelagem rica do domínio com Aggregates, Entities, Value Objects e Domain Events
- CQRS (Command Query Responsibility Segregation): separação explícita entre operações de escrita (Commands) e leitura (Queries)
- Event-Driven: eventos de domínio propagados via RabbitMQ para desacoplamento e resiliência
- API-first: contratos REST bem definidos com versionamento, webhooks e documentação OpenAPI
- Multi-tenant by design: isolamento de dados nativo com TenantId em todas as entidades
- 100% Open-Source: todas as tecnologias com licença gratuita (Apache 2.0, MIT, MPL)

1.2. Stack Tecnológico Decidido

Camada	Tecnologia	Licença
Linguagem	C# / .NET 8+ (LTS)	MIT
Escrita/CRUD	PostgreSQL + EF Core	PostgreSQL License + Apache 2.0
Leitura/Relatórios	PostgreSQL + Dapper	Apache 2.0
Mensageria	RabbitMQ	MPL 2.0 (gratuito)
Mediator/CQRS	Mediator (source generators)	Apache 2.0
Mapping	Mapperly (compile-time)	Apache 2.0
Validação	FluentValidation	Apache 2.0
Testes	xUnit + FluentAssertions + NSubstitute + Testcontainers	Apache 2.0 / MIT
CI/CD	GitHub Actions	Gratuito (repos públicos)

2. Estrutura de Camadas (Agente Arquitetura)

A API segue Clean Architecture com 4 camadas. A regra fundamental: camadas internas nunca referenciam camadas externas. A dependência é sempre de fora para dentro, via interfaces definidas no Domain ou Application.

2.1. Diagrama de Dependências

```

AgendeAqui.Api (Presentation) → depende de Application
AgendeAqui.Application          → depende de Domain
AgendeAqui.Domain               → NÃO depende de NADA
AgendeAqui.Infrastructure       → implementa interfaces do Domain/Application

```

2.2. Domain Layer (AgendeAqui.Domain)

Coração do sistema. Contém toda a lógica de negócio, sem nenhuma dependência externa. Zero referências a frameworks, ORM ou bibliotecas de infraestrutura.

Conteúdo da camada:

- Entities: Appointment, Professional, Service, Client, Schedule, Tenant
- Value Objects: TimeSlot, PhoneNumber, Email, TenantId, AppointmentStatus
- Aggregates: Appointment (raiz), Schedule (raiz de disponibilidade)
- Domain Events: AppointmentCreatedEvent, AppointmentCancelledEvent, AppointmentRescheduledEvent
- Repository Interfaces: IAppointmentRepository, IScheduleRepository, IProfessionalRepository
- Service Interfaces: INotificationProvider, IAvailabilityChecker
- Domain Exceptions: TimeSlotConflictException, ProfessionalUnavailableException

Regra de ouro:

PROIBIDO no Domain: using Microsoft.EntityFrameworkCore, using RabbitMQ, using qualquer pacote NuGet de infraestrutura. Apenas System. e interfaces próprias.*

2.3. Application Layer (AgendeAqui.Application)

Orquestra os casos de uso. Contém Commands, Queries, Handlers, DTOs e Validators. Depende apenas do Domain. Usa Mediator para despacho de commands/queries e FluentValidation para validação.

Separação CQRS:

- Commands (escrita): CreateAppointmentCommand, CancelAppointmentCommand, RescheduleAppointmentCommand, MarkAttendanceCommand
- Queries (leitura): GetAvailableSlotsQuery, GetAppointmentsByDateQuery, GetAttendanceReportQuery, GetProfessionalScheduleQuery
- Handlers: cada Command/Query tem seu Handler dedicado (Single Responsibility)
- Validators: CreateAppointmentValidator, RescheduleAppointmentValidator (FluentValidation)
- DTOs/ViewModels: AppointmentDto, AvailableSlotDto, AttendanceReportDto
- Interfaces de Serviço: IAppointmentService, INotificationService, IReportService

Pipeline Behaviors (cross-cutting concerns via Mediator):

- ValidationBehavior: intercepta todo command/query e executa FluentValidation antes do handler
- LoggingBehavior: structured logging de entrada/saída de cada request
- TenantBehavior: injeta o TenantId no contexto de cada operação

2.4. Infrastructure Layer (AgendeAqui.Infrastructure)

Implementa as interfaces definidas nas camadas internas. Contém acesso a banco de dados, mensageria, serviços externos e configurações.

Sub-projetos:

- Persistence: DbContext (EF Core), Migrations, Configurations (Fluent API), Repositories, UnitOfWork
- Messaging: RabbitMQ producers/consumers, IntegrationEventBus, event serialization
- Notifications: implementações de INotificationProvider (WhatsAppProvider, SmsProvider, EmailProvider)
- ExternalServices: clients para WhatsApp BSP (Chakra Chat/Zenvia), Google Calendar sync

Decisão de persistência dual:

- EF Core para Commands (escrita): change tracking, migrations, validações de integridade, transaction management
- Dapper para Queries (leitura): SQL otimizado, sem overhead de tracking, ideal para relatórios de presença e dashboards

2.5. Presentation Layer (AgendeAqui.Api)

Camada fina que expõe endpoints REST. Controllers delegam tudo para o Mediator. Sem lógica de negócio.

Responsabilidades:

- Controllers: recebem HTTP request, criam Command/Query, enviam via Mediator, retornam ActionResult
- Middleware: autenticação JWT, tratamento global de exceções, tenant resolution (header X-Tenant-Id)
- Filters: rate limiting por tenant, request/response logging
- Swagger/OpenAPI: documentação automática dos contratos de API

3. Estrutura da Solution (Agente .NET)

3.1. Organização de Projetos

```
AgendeAqui/
├── src/
│   ├── AgendeAqui.Domain/           ← Class Library (.NET 8)
│   ├── AgendeAqui.Application/      ← Class Library (.NET 8)
│   └── AgendeAqui.Infrastructure/    ← Class Library (.NET 8)
```

```
| └─ AgendeAqui.Api/                ← ASP.NET Core Web API (.NET 8)
| └─ tests/
|   └─ AgendeAqui.Domain.Tests/    ← Testes unitários
|   └─ AgendeAqui.Application.Tests/ ← Testes de handlers
|   └─ AgendeAqui.Infrastructure.Tests/ ← Testes de integração
|   └─ AgendeAqui.Api.Tests/       ← Testes E2E
└─ docker-compose.yml              ← PostgreSQL + RabbitMQ + API
```

3.2. Mediator (Source Generators) — por que não MediatR

O MediatR migrou para licença comercial (RPL-1.5) a partir da versão 13. A biblioteca Mediator (por Martin Othamar) é open-source (Apache 2.0), usa source generators para gerar o código em compile-time, e oferece performance até 2x superior por eliminar reflection. API quase idêntica ao MediatR.

Vantagens do Mediator:

- Zero reflection: source generators geram todo o dispatch em compile-time
- ValueTask<T>: menos alocações em handlers síncronos
- Diagnósticos em compile-time: se um handler não existe, o build falha (não explode em runtime)
- Native AOT ready: compatível com publicação AOT do .NET 8+
- Apache 2.0: licença gratuita para uso comercial, sem restrições

3.3. Mapperly (Compile-Time Mapping) — por que não AutoMapper

O AutoMapper também migrou para licença comercial junto com o MediatR. Mapperly gera código de mapeamento via source generators em compile-time, eliminando overhead de runtime e garantindo type-safety no build.

- Erros de mapeamento detectados em compile-time (não em runtime)
- Zero reflection, zero alocações adicionais
- Performance equivalente a mapeamento manual
- Apache 2.0: 100% gratuito

4. Estratégia Multi-Tenant (Agente PostgreSQL)

4.1. Decisão: Row-Level com TenantId + RLS

Após análise das três estratégias possíveis (database-per-tenant, schema-per-tenant, row-level), a recomendação é row-level com TenantId, reforçado por Row-Level Security (RLS) do PostgreSQL.

Estratégia	Isolamento	Custo/Complexidade	Escala 500+ tenants
Database-per-tenant	Máximo	Muito alto	Inviável
Schema-per-tenant	Alto	Alto	Problemático
Row-level + RLS ☒	Bom (DB enforced)	Baixo	Excelente

Como funciona:

1. Toda tabela contém coluna TenantId (UUID, NOT NULL, indexada)
2. PostgreSQL RLS (Row-Level Security) ativado em todas as tabelas com dados de tenant
3. Middleware da API resolve o TenantId (via JWT claim ou header) e seta session variable: SET app.current_tenant = '<tenant-id>'
4. Policy RLS filtra automaticamente: CREATE POLICY tenant_isolation ON appointments USING (tenant_id = current_setting('app.current_tenant')::uuid)
5. EF Core Global Query Filter como segunda camada: .HasQueryFilter(e => e.TenantId == _tenantContext.TenantId)

Defesa em profundidade: mesmo que a aplicação tenha um bug que esqueça o filtro, o PostgreSQL RLS impede vazamento de dados entre tenants no nível do banco.

4.2. Modelagem do Banco de Dados

Entidades principais:

- tenants: id, name, slug, whatsapp_number, plan_type, is_active, created_at
- professionals: id, tenant_id, name, email, phone, specialties[], is_active
- services: id, tenant_id, name, duration_minutes, price, is_active
- schedules: id, tenant_id, professional_id, day_of_week, start_time, end_time, is_recurring
- appointments: id, tenant_id, professional_id, service_id, client_id, scheduled_at, status (scheduled/confirmed/completed/cancelled/no_show), notes
- clients: id, tenant_id, name, phone, email, whatsapp_opt_in
- notifications: id, tenant_id, appointment_id, type (confirmation/reminder_24h/reminder_2h/cancellation), channel (whatsapp/sms/email), status, sent_at
- attendance_logs: id, tenant_id, appointment_id, checked_in_at, checked_in_by, status (present/absent/late)

Índices críticos para performance:

- B-tree index em tenant_id em TODAS as tabelas (obrigatório para RLS performar bem)
- Composite index: (tenant_id, professional_id, scheduled_at) em appointments — consulta de disponibilidade
- Composite index: (tenant_id, scheduled_at, status) em appointments — relatórios de presença
- Partial index: WHERE status = 'scheduled' em appointments — otimiza consultas de agendamentos futuros

5. Mensageria e Eventos (Agente Mensageria)

5.1. Arquitetura Event-Driven com RabbitMQ

O RabbitMQ é o message broker central da arquitetura. Todos os eventos de domínio e integração fluem pelo RabbitMQ, garantindo desacoplamento entre a API e os consumidores (notificações, relatórios, integrações externas).

Exchanges e Filas:

Exchange	Tipo	Filas Vinculadas	Propósito
appointment.events	Topic	notifications.queue, reports.queue, integrations.queue	Eventos de agendamento (created, cancelled, rescheduled)
notification.events	Direct	whatsapp.queue, sms.queue, email.queue	Roteamento por canal de notificação
scheduling.dlx	Fanout	dead-letter.queue	Mensagens que falharam após retries

5.2. Fluxo de Notificações

1. Appointment é criado via Command → handler persiste no PostgreSQL
1. Domain Event AppointmentCreatedEvent é publicado via Mediator (in-process)
2. Integration Event handler converte para IntegrationEvent e publica no RabbitMQ
3. notification.consumer recebe o evento e despacha para o canal correto (WhatsApp/SMS/Email)
4. WhatsApp Provider (Chakra Chat API) envia a template message de confirmação
5. Scheduled messages (lembretes 24h e 2h): um background worker (Hosted Service) consulta agendamentos próximos e publica eventos de lembrete no RabbitMQ

Abstração de Notificações:

Interface INotificationProvider com implementações intercambiáveis: WhatsAppProvider (Chakra Chat), SmsProvider (Twilio/Vonage), EmailProvider (SendGrid/SMTP). Trocar o provider não impacta o core.

5.3. Retry e Dead Letter

- Retry policy: 3 tentativas com backoff exponencial (1s, 5s, 25s)
- Dead Letter Queue (DLQ): mensagens que falharam após 3 tentativas vão para scheduling.dlx
- Monitoramento: dashboard do RabbitMQ Management UI + alertas quando DLQ > 0
- Idempotência: cada mensagem carrega um MessageId único; consumers verificam antes de processar

6. Conexões com APIs e Serviços Externos

6.1. Mapa de Integrações

Serviço	Direção	Protocolo	Propósito
WhatsApp BSP (Chakra Chat)	API → Externo	REST API + Webhooks	Envio de notificações e recebimento de mensagens
Chatbot WhatsApp	Externo → API	REST API	Agendamento via conversação
VISU (sistema legado)	Externo → API	REST API	CRUD de agendamentos pelo atendente
Google Calendar	Bidirecional	REST API + Webhooks	Sincronização de agenda
Gateway Pagamento	API → Externo	REST API + Webhooks	Cobrança por uso / assinatura
n8n / Zapier	Externo → API	Webhooks + REST	Automações customizadas
ERPs Verticais	Externo → API	REST API	Módulo de agendamento embarcado

6.2. Padrão de Conexão: API REST + Webhooks

Todos os consumidores externos se conectam à API AgendeAqui através de endpoints REST padronizados. Para eventos assíncronos (confirmação, cancelamento, lembrete), a API dispara webhooks registrados pelo tenant.

Endpoints principais da API v1:

- POST /api/v1/appointments — criar agendamento
- GET /api/v1/appointments/{id} — consultar agendamento
- PUT /api/v1/appointments/{id}/reschedule — reagendar
- PUT /api/v1/appointments/{id}/cancel — cancelar
- PUT /api/v1/appointments/{id}/attendance — registrar presença/ausência
- GET /api/v1/availability?professionalId=X&date=Y — consultar horários livres
- GET /api/v1/reports/attendance?from=X&to=Y — relatório de presença
- POST /api/v1/webhooks — registrar webhook do tenant

Autenticação e Autorização:

- JWT Bearer token com claims: tenant_id, role (admin/professional/client)
- API Key para integrações machine-to-machine (chatbots, ERPs)
- Rate limiting por tenant: 100 req/min (Starter), 1000 req/min (Business)

7. Estratégia de Testes (Agente QA)

7.1. Pirâmide de Testes

Nível	Ferramenta	Escopo	Quantidade
Unitários	xUnit + NSubstitute	Entities, Value Objects, Handlers	~70% dos testes
Integração	Testcontainers + EF Core	Repositories, DB, RabbitMQ	~20% dos testes
E2E / API	WebApplicationFactory	Endpoints REST completos	~10% dos testes

7.2. Testcontainers para Testes de Integração

Testcontainers sobe instâncias reais de PostgreSQL e RabbitMQ em containers Docker durante os testes, garantindo que os testes de integração rodam contra a mesma infraestrutura de produção.

- PostgreSQL container: testa migrations, queries Dapper, RLS policies, constraints
- RabbitMQ container: testa publish/consume de eventos, DLQ, retry policies
- Cleanup automático: cada teste roda em estado limpo (database por test class)

7.3. Testes de Arquitetura

Testes automatizados que validam as regras de Clean Architecture em compile-time:

- Domain não pode referenciar Infrastructure ou Application
- Application não pode referenciar Infrastructure
- Controllers não podem conter lógica de negócio (apenas despacho via Mediator)
- Ferramenta: NetArchTest ou ArchUnitNET (ambas open-source)

8. Disponibilidade e Observabilidade (Agente Disponibilidade)

8.1. Meta de SLA: 99,5%

Derivado do requisito de negócio: lembretes de agendamento não podem falhar. 99,5% de uptime permite no máximo ~3,65 horas de downtime por mês.

Estratégias de resiliência:

- Health checks: /health (liveness) e /health/ready (readiness) verificando PostgreSQL + RabbitMQ
- Circuit breaker: Polly para chamadas ao WhatsApp BSP (open after 5 failures, half-open after 30s)
- Retry com backoff: chamadas HTTP externas com 3 retries exponenciais
- Graceful degradation: se WhatsApp falhar, fallback para SMS; se SMS falhar, fallback para email

- Queue durability: RabbitMQ com filas duráveis e mensagens persistentes (sobrevive a restart)

8.2. Observabilidade

- Structured Logging: Serilog com sinks para console + file (JSON) + Seq/Grafana Loki
- Métricas: Prometheus exporter com métricas por tenant (agendamentos/min, latência p95, erros)
- Health Dashboard: Grafana com dashboards de SLA, RabbitMQ queue depth, PostgreSQL connections
- Alertas: alerta quando DLQ > 0, quando latência p95 > 200ms, quando error rate > 1%
- Tracing: OpenTelemetry para rastrear request completo (API → Handler → DB → RabbitMQ → Notification)

8.3. Requisitos Não-Funcionais

Requisito	Meta	Como Medir
Disponibilidade	99,5%	Uptime monitoring + health checks
Latência (disponibilidade)	<200ms p95	Prometheus histogram
Latência (CRUD)	<500ms p95	Prometheus histogram
Escalabilidade	500+ tenants, 5K agend./mês cada	Load testing com k6
Segurança	LGPD compliant	Auditoria + criptografia at rest

9. Plano de Implementação Incremental

9.1. Sprint 1-2: Foundation

- Criar solution com 4 projetos (Domain, Application, Infrastructure, Api)
- Configurar Docker Compose (PostgreSQL 16 + RabbitMQ 3.13)
- Implementar Tenant entity, middleware de resolução de tenant, RLS no PostgreSQL
- Configurar Mediator + Mapperly + FluentValidation
- CI: GitHub Actions com build + testes unitários

9.2. Sprint 3-4: Core Scheduling

- Implementar Aggregates: Appointment, Schedule, Professional, Service
- Commands: CreateAppointment, CancelAppointment, RescheduleAppointment
- Queries: GetAvailableSlots, GetAppointmentsByDate (Dapper)
- Testes unitários do Domain + Testes de integração com Testcontainers

9.3. Sprint 5-6: Notifications

- RabbitMQ: exchanges, filas, producers/consumers
- INotificationProvider: implementação WhatsApp (Chakra Chat)
- Background Worker: scheduled reminders (24h e 2h antes)
- DLQ + retry + idempotência

9.4. Sprint 7-8: Reports + Polish

- MarkAttendance Command + GetAttendanceReport Query (Dapper)
- Webhooks: registro e disparo de eventos para integrações
- Swagger/OpenAPI + documentação de contratos
- Testes E2E + load testing com k6
- Observabilidade: Serilog + Prometheus + Health Checks

10. Conclusão

O planejamento arquitetural confirma que a API AgendeAqui pode ser construída com uma stack 100% open-source, seguindo Clean Architecture + DDD + CQRS, com isolamento multi-tenant robusto (RLS no PostgreSQL) e comunicação event-driven (RabbitMQ). A separação entre EF Core (escrita) e Dapper (leitura) garante performance para relatórios sem comprometer a integridade das operações de escrita.

Próximo passo: com a arquitetura planejada, avançar para a implementação do Sprint 1 — criando a solution, configurando Docker Compose, e implementando a foundation de multi-tenancy com RLS.